

JUnit Testing and Code Coverage Analysis Report for Point of Sale (POS) System

KASHAF FATIMA

✉ kashaf.fatimaa132@gmail.com

☎ +92 3466208273

Contents

Introduction	2
a. Report Table:	2
b. Explanation:	13
1. Statement Coverage	13
2. Decision Coverage	16
3. Condition Coverage	19
a. Generate and Submit Reports:	20

Introduction

This report presents a comprehensive software testing and code coverage analysis conducted on a Point of Sale (POS) System using JUnit testing and EclEmma coverage analysis tools. The objective of this project was to evaluate the correctness, reliability, and quality of various system components through systematic unit testing and coverage assessment.

The testing process involved the execution of test cases across multiple classes, including POS, PointOfSale, POSSystem, Inventory, Employee, Management, Item, ReturnItem, and Employee Management modules. Test results were analyzed to identify passed and failed scenarios, assess software behavior under different conditions, and evaluate the effectiveness of the implemented functionality.

In addition to functional verification, statement coverage, decision coverage, and condition coverage analyses were performed to measure the extent to which the source code was exercised during testing. Areas where full coverage was achieved, coverage limitations, and recommendations for improving test effectiveness are documented within this report.

The report also includes JUnit execution results, coverage reports generated using EclEmma, identified coverage gaps, and proposed improvements aimed at enhancing software quality, maintainability, and reliability. This project demonstrates practical experience in software testing, quality assurance, unit test development, defect identification, and code coverage evaluation using industry-standard testing tools and methodologies.

a. Report Table:

1. POS Test Cases Summary

TestID	Testcase name	Testcase description	Class	Function Being Tested	Test Inputs	Expected Output	Actual Output	Pass/Fail Status
TC01	Delete TempItem	Tests deleting a temp item from the file	POS	deleteTempItem	itemID=1005	Temp file does not contain 1005	As Expected	Pass
TC02	testEndPOS_SumCheck	Tests the sum check for ending POS	POS	endPOS	file=ITEM_DATABASE_FILE	Total price = 0.0	As Expected	Pass
TC03	testEndPOS_ReturnSaleUpdate	Tests if return sale updates	POS	endPOS	file=ITEM_DATABASE_FILE	Return sale contains	As Expected	Pass

		are correctly written				updated entry		
TC04	testEndPOS_ItemDatabaseUpdate	Tests if item database is updated correctly	POS	endPOS	file=ITEM_DATABASE_FILE	Item database does not contain 1005	As Expected	Pass
TC05	testRetrieveTemp_AmountCheck1005	Tests if amount is correct after retrieving temp (1005)	POS	retrieveTemp	file=ITEM_DATABASE_FILE	Amount for item 1005 is 3	As Expected	Fail
TC06	testRetrieveTemp_AmountCheck1002	Tests if amount is correct after retrieving temp (1002)	POS	retrieveTemp	file=ITEM_DATABASE_FILE	Amount for item 1002 is 2	As Expected	Fail
TC07	testRetrieveTemp_TotalPriceUpdate	Tests if total price is updated after retrieving temp	POS	retrieveTemp	file=ITEM_DATABASE_FILE	Total price > 0	As Expected	Fail
TC08	testDefaultConstructor	Tests default constructor of POS class	POS	POS (constructor)	N/A	No errors or exceptions	As Expected	Pass

2. Test Cases Summary

TestID	Testcase name	Testcase description	Class	Function Being Tested	Test Inputs	Expected Output	Actual Output	Pass/Fail Status
TC01	testStartNew	Tests starting a new transaction	PointOf Sale	startNew	DATABASE_FILE	True, database item populated	As Expected	Pass
TC02	testEnterItem	Tests entering a valid item	PointOf Sale	enterItem	itemID=1005, amount=3	True, item added to transaction	As Expected	Fail
TC03	testEnterItemItemNotFound	Tests entering an invalid item	PointOf Sale	enterItem	invalidItemID=9999, amount=2	False, no item added	As Expected	Pass
TC04	testUpdateTotal	Tests updating total after	PointOf Sale	updateTotal	itemID=1005, amount=2	Updated total	As Expected	Fail

		item addition				calculated correctly		
TC05	testUpdateTotalWithNoItems	Tests total update with no items	PointOf Sale	updateTotal	No items	Total remains unchanged	As Expected	Fail
TC06	testValidCoupon	Tests applying a valid coupon	PointOf Sale	coupon	validCoupon=C005	Total price reduced by 10%	As Expected	Pass
TC07	testInvalidCoupon	Tests applying an invalid coupon	PointOf Sale	coupon	invalidCoupon=INVALID_COUPON	Total price unchanged	As Expected	Pass
TC08	testCouponFileNotFound	Tests applying a coupon with missing file	PointOf Sale	coupon	validCoupon=C001	False, coupon not applied	As Expected	Pass
TC09	testCreateTemp	Tests temporary item creation	PointOf Sale	createTemp	itemID=1010, amount=5	Temp file contains item details	As Expected	Pass
TC10	testRemoveItems	Tests removing items from cart and temp file	PointOf Sale	removeItems	itemIDToRemove=1010	Item removed from cart and temp	As Expected	Pass
TC11	testCreditCard	Tests credit card validation	PointOf Sale	creditCard	creditCardNumber=1234567812345678	Valid card returns true, invalid returns false	As Expected	Fail
TC12	testLastAddedItem	Tests retrieval of last added item	PointOf Sale	lastAddedItem	No input	Returns last added item details	As Expected	Fail
TC13	testGetCart	Tests retrieving the cart items	PointOf Sale	getCart	No input	Cart contains correct items	As Expected	Pass
TC14	testGetCartSize	Tests retrieving the cart size	PointOf Sale	getCartSize	No input	Returns correct cart size	As Expected	Fail

TC15	testGetTotal	Tests retrieving the total price	PointOf Sale	getTotal	No input	Returns correct total price	As Expected	Fail
------	--------------	----------------------------------	--------------	----------	----------	-----------------------------	-------------	------

3. Return Item Test Cases Summary

TestID	Testcase name	Testcase description	Class	Function Being Tested	Test Inputs	Expected Output	Actual Output	Pass/Fail Status
TC01	testGetItemID_10001	Tests getting item ID for item 10001	ReturnItem	getItemID	itemID=10001, days=12	10001	As Expected	Pass
TC02	testGetItemID_10002	Tests getting item ID for item 10002	ReturnItem	getItemID	itemID=10002, days=12	10002	As Expected	Pass
TC03	testGetDays_12	Tests getting days value (12 days)	ReturnItem	getDays	itemID=10001, days=12	12	As Expected	Pass
TC04	testGetDays_5	Tests getting days value (5 days)	ReturnItem	getDays	itemID=10001, days=5	5	As Expected	Pass

4. Test Cases and Coverage

TestID	Testcase Name	Testcase Description	Class	Function Being Tested	Test Inputs	Expected Output	Actual Output	Pass/Fail Status
1	testContinueFromTemp_Return	Test continueFromTemp function for Return scenario	POSSystem	continueFromTemp	1234567892L	Return	Return	Pass
2	testContinueFromTemp_Sale	Test continueFromTemp function for Sale scenario	POSSystem	continueFromTemp	1234567892L	Sale	Sale	Pass

3	testContinueFromTemp_Rental	Test continueFromTemp function for Rental scenario	POSSystem	continueFromTemp	1234567892L	Rental	Rental	Fail
4	testCheckTemp_True	Test checkTemp function when temp file exists	POSSystem	checkTemp	None	True	True	Fail
5	testCheckTemp_False	Test checkTemp function when temp file does not exist	POSSystem	checkTemp	None	False	False	Pass
6	testLogIn_ValidCashier	Test logIn function with valid cashier credentials	POSSystem	logIn	110002, lehigh2016	1	1	Fail
7	testLogIn_ValidAdmin	Test logIn function with valid admin credentials	POSSystem	logIn	110001, 1	1	1	Pass
8	testLogIn_InvalidUsername	Test logIn function with invalid username	POSSystem	logIn	112, lehigh2016	0	0	Pass
9	testLogIn_InvalidPassword	Test logIn function with invalid password	POSSystem	logIn	110002, lehig	0	0	Pass
10	testLogOut	Test logOut function	POSSystem	logOut	Cashier	None	None	Pass

5. Test Cases and Coverage - PORTest

TestID	Testcase Name	Testcase Description	Class	Function Being Tested	Test Inputs	Expected Output	Actual Output	Pass/Fail Status
1	testDeleteTempltem_1005	Test deleteTempltem	POR	deleteTempltem	1005	Does not contain 1005,	Does not contain 1005, contains 1002	Pass

		function to delete item 1005				contains 1002		
2	testEndPOS_ItemDatabase	Test endPOS function to update itemDatabase and returnSale files	POR	endPOS	ITEM_DATABASE_FILE	0.0, updates in returnSale and itemDatabase files	0.0, updates in returnSale and itemDatabase files	Pass
3	testRetrieveTemp_1005	Test retrieveTemp function to retrieve item 1005	POR	retrieveTemp	ITEM_DATABASE_FILE	transactionItem[0].amount == 3	transactionItem[0].amount == 3	Fail
4	testRetrieveTemp_1002	Test retrieveTemp function to retrieve item 1002	POR	retrieveTemp	ITEM_DATABASE_FILE	transactionItem[1].amount == 2, totalPrice > 0	transactionItem[1].amount == 2, totalPrice > 0	Fail
5	testDefaultConstructor	Test default constructor with phoneNum == 0	POR	Constructor	0	phoneNum == 0	phoneNum == 0	Pass

6. Test Cases and Coverage - EmployeeTest

TestID	Testcase Name	Testcase Description	Class	Function Being Tested	Test Inputs	Expected Output	Actual Output	Pass/Fail Status
1	testGetUsername_10001	Test getUsername function for Employee 10001	Employee	getUsername	10001, Alpha Release, Admin, 123345	10001	10001	Pass
2	testGetName_10002	Test getName function for	Employee	getName	10002, Alpha	Alpha Release	Alpha Release	Pass

		Employee 10002			Release, Admin, 123345			
3	testGetPosition_10003	Test getPosition function for Employee 10003	Employee	getPosition	10003, Alpha Release, Admin, 123345	Admin	Admin	Pass
4	testGetPassword_10004	Test getPassword function for Employee 10004	Employee	getPassword	10004, Alpha Release, Admin, 123345	123345	123345	Pass
5	testSetName_10005	Test setName function for Employee 10005	Employee	setName	10005, Beta Release, Admin, 123345	Beta Release	Beta Release	Pass
6	testSetPosition_10006	Test setPosition function for Employee 10006	Employee	setPosition	10006, Alpha Release, QA, 123345	QA	QA	Pass
7	testSetPassword_10007	Test setPassword function for Employee 10007	Employee	setPassword	10007, Alpha Release, Admin	54321	54321	Pass

					n, 54321			
--	--	--	--	--	-------------	--	--	--

7. Test Case Summary- Employee Management

TestID	Testcase name	Testcase description	Class	Function Being Tested	Test Inputs	Expected Output	Actual Output	Pass/Fail Status
1	testAdd_Cashier	Test to verify cashier addition	Employee Management	add	"Alpha Release", "123345", true	Employee added as cashier	Employee added as cashier	Pass
2	testAdd_Admin	Test to verify admin addition	Employee Management	add	"Alpha Release2", "123345", false	Employee added as admin	Employee added as admin	Pass
3	testDelete_ExistingEmployee	Test to delete an existing employee	Employee Management	delete	"DeleteTest Employee"	Employee deleted successfully	Employee deleted successfully	Pass
4	testDelete_NonExistingEmployee	Test to delete a non-existing employee	Employee Management	delete	"Nonexistent employee"	Employee not deleted	Employee not deleted	Pass
5	testUpdate_ValidUpdate	Test to update employee with valid details	Employee Management	update	"UpdateTest Employee", "newPass456", "Admin", "UpdatedTest Employee"	Employee details updated	Employee details updated	Pass
6	testUpdate_InvalidUsername	Test to update with invalid username	Employee Management	update	"99999", "newPass456", "Admin", "Nonexistent Employee"	Error: Employee not found	Error: Employee not found	Pass
7	testUpdate_InvalidPosition	Test to update with invalid position	Employee Management	update	"PositionTest Employee", "newPass456", "InvalidPosition", "UpdatedTest Employee"	Error: Invalid position	Error: Invalid position	Pass

8. Test Case Summary - InventoryTest

Test ID	Testcase name	Testcase description	Class	Function Being Tested	Test Inputs	Expected Output	Actual Output	Pass/Fail Status
1	testGetInstance	Test singleton pattern for inventory	Inventory	getInstance	None	Same instance returned	Same instance returned	Pass
2	testGetInstanceIsNotNull	Test that inventory instance is not null	Inventory	getInstance	None	Instance is not null	Instance is not null	Pass
3	testAccessInventory	Test access to inventory from file	Inventory	accessInventory	tempDatabaseFile, databaseItem	Inventory accessed, 101 items loaded	Inventory accessed, 101 items loaded	Pass
4	testUpdateInventorySale_Potato	Test inventory sale update for Potato	Inventory	updateInventory	tempDatabaseFile, transactionItem, databaseItem, true	Potato inventory reduced by 49	Potato inventory reduced by 49	Pass
5	testUpdateInventorySale_PlasticCup	Test inventory sale update for Plastic Cup	Inventory	updateInventory	tempDatabaseFile, transactionItem, databaseItem, true	Plastic Cup inventory reduced by 100	Plastic Cup inventory reduced by 100	Pass
6	testUpdateInventoryReturn_SkirtSteak	Test inventory return update for Skirt Steak	Inventory	updateInventory	tempDatabaseFile, transactionItem, databaseItem, false	Skirt Steak inventory increased by 55	Skirt Steak inventory increased by 55	Pass
7	testUpdateInventoryReturn_PotatoChips	Test inventory return update for Potato Chips	Inventory	updateInventory	tempDatabaseFile, transactionItem, databaseItem, false	Potato Chips inventory increased by 32	Potato Chips inventory increased by 32	Pass

9. Test Case Summary - ItemTest

Test ID	Testcase name	Testcase description	Class	Function Being Tested	Test Inputs	Expected Output	Actual Output	Pass/Fail Status
1	testItemName	Test to check item name	Item	getItemName	Item(1102, "Test item", 100, 123)	"Test item"	"Test item"	Pass
2	testItemID	Test to check item ID	Item	getItemID	Item(1103, "Test item", 100, 123)	1103	1103	Pass
3	testItemPrice	Test to check item price	Item	getItemPrice	Item(1104, "Test item", 100, 123)	100	100	Pass
4	testAmount	Test to check item amount	Item	getItemAmount	Item(1105, "Test item", 100, 123)	123	123	Pass
5	testupdateAmount	Test to update item amount	Item	updateAmount	Item(1105, "Test item", 100, 123), updateAmount(150)	150	150	Pass

10. Test Cases Summary

Test ID	Testcase Name	Testcase Description	Class	Function Being Tested	Test Inputs	Expected Output	Actual Output	Pass /Fail Status
1	testDeleteTemplItem_1005	Test deletion of item with ID 1005 from temp file	POH	deleteTemplItem	1005	Item 1005 should be removed from temp file	Item 1005 is removed	Pass
2	testDeleteTemplItem_1002	Test deletion of item with ID 1002 from temp file	POH	deleteTemplItem	1002	Item 1002 should be removed from temp file	Item 1002 is removed	Pass
3	testEndPOS_Correct Total	Test if the total price is calculated correctly at the end of POS	POH	endPOS	ITEM_DATABASE_FILE	Total price should be 0.0	Total price is 0.0	Fail

4	testEndPOS_UpdateReturnSale	Test if the return sale file is updated correctly after POS ends	POH	endPOS	ITEM_DATAB ASE_FILE	Return sale file should contain updated information for item 1002	Return sale file contains correct update	Fail
5	testEndPOS_UpdateItemDatabase	Test if the item database is updated correctly after POS ends	POH	endPOS	ITEM_DATAB ASE_FILE	Item 1005 should be removed from the database	Item 1005 is removed	Pass
6	testRetrieveTemp_Item1005	Test retrieval of temporary item 1005	POH	retrieveTemp	ITEM_DATAB ASE_FILE	Item 1005 should have 3 units in the transaction list	Item 1005 has 3 units	Pass
7	testRetrieveTemp_Item1002	Test retrieval of temporary item 1002	POH	retrieveTemp	ITEM_DATAB ASE_FILE	Item 1002 should have 2 units in the transaction list and total price > 0	Item 1002 has 2 units, total price > 0	Fail
8	testDefaultConstructor	Test the default constructor of POH class	POH	POH (default constructor)	None	POH phone should be initialized to 0	POH phone is 0	Fail

11. Test Cases Summary for ManagementTest

Test ID	Testcase Name	Testcase Description	Class	Function Being Tested	Test Inputs	Expected Output	Actual Output	Pass /Fail Status
1	testCheckUser_UserExists	Test if checkUser correctly identifies an existing user	Management	checkUser	6096515668L	True	True	Pass
2	testCheckUser_UserDoesNotExist	Test if checkUser correctly identifies a non-existing user	Management	checkUser	12345678L	False	False	Pass

3	testGetLatestReturnDate_ValidPhone	Test retrieval of latest return date for a valid phone	Management	getLatestReturnDate	1112223334L	Return list with 1 item, days > 0	Correct return item, days > 0	Pass
4	testCreateUser_ValidPhone	Test if a user is correctly created in the database	Management	createUser	1234567890L	True, user found in file	True, user found	Pass
5	testAddRental_SingleItem	Test if a single item is added correctly for a user	Management	addRental	1234567890L, Item(1111)	Rental added for the user	Rental found in file	Pass
6	testAddRental_MultipleItems	Test if multiple items are added correctly for a user	Management	addRental	1234567890L, Item(1111, 1112)	Rentals added for the user	Rentals found in file	Pass
7	testUpdateRentalStatus_SingleItem	Test if the rental status is updated for a single returned item	Management	updateRentalStatus	9876543455L, ReturnItem(1115, 2)	Rental status updated in file	Rental status updated	Pass
8	testUpdateRentalStatus_MultipleItems	Test if the rental status is updated for multiple returned items	Management	updateRentalStatus	9876543455L, ReturnItem(1115, 1116)	Rental statuses updated in file	Rental statuses updated	Pass

b. Explanation:

1. Statement Coverage

Coverage Achieved: Explain where and why 100% coverage was possible.

1. ReturnItem Class

The fact that the ReturnItem class is composed in a very transparent manner, it is easy to get 100% statement coverage. On one hand, there are two getter functions for the instance variables itemID and daysSinceReturn; that directly return their values and, of course, the constructor. The class contains no loop or conditional statements. Thus, the only lines of code not executed in the testing of the constructor and two getter functions are those which are guaranteed to be covered - hence, in total, 100% statement coverage is achieved.

2. Item Class:

Given the clarity of the ReturnItem class, it also is easy to obtain a statement coverage 100%. The instance variables itemID and daysSinceReturn have two getter methods that directly return the values of these variables along with a constructor. There are no loops or conditional statements within the class. Thus, all the lines of code will be executed as there is testing of the constructor as well as both getter functions with which the statement coverage will be ensured to be complete.

3. Employee Class:

Because the Employee class makes uses of simple setter and getter methods for defining and retrieving attributes, it can also easily achieve 100% statement coverage. The getter methods return values for the four parameters the constructor was using to initialize the object: password, username, position, and name. Again, the setter methods don't have any branching logic or conditions but could change the values of the attributes. All statements of the class will be executed in case testing of the constructor, each getter function and each setter method. Hence this guarantees full statement coverage.

Coverage Gaps: Describe any areas where 100% coverage was not achievable, including reasons for these gaps (e.g., limitations in the code, untestable scenarios).

4. login() in POSSystem

The `login` method has various disadvantages that make it impossible to score 100% coverage. A major drawback is that the same result, namely 0 is given when providing either a wrong username and/or a wrong password; therefore tests cannot distinguish between both cases of failure. In addition, if there exist other jobs with their positions whose roles are not particularly confirmed in the requirements (those jobs other than "Cashier" or "Admin"), then the outcome of its reason may be dependent on their position that may lead to untested branches. It is also hard to replicate the side effects of writing to a log file in test environment for they are complex. These factors together prevent complete coverage of all possible cases and leave some of them untestable.

5. retrieveTemp in POS

Some untestable conditions in `retrieveTemp`-method make some lines not covered at all. When the `tempFile` is not there, it throws `FileNotFoundException` and those code-statements which read from the file are not executed that line of the code-not tested, the same happens on further lines, processing the contents of the file-that also get skipped as it has an exception while reading the file. But the loop checking remaining lines will not run at all if the first read returns {null}, which would mean those statements are not executed; hard to get full statement coverage in such cases as they result in coverage gaps.

6. updateTotal in PointOfSale

Since not all possible conditional paths are covered by these existing tests, the updateTotal technique doesn't achieve 100% branch coverage. The most dangerous situation here is an exception being thrown because the transactionItem list is empty. Since this case is inadequately addressed or tested, the corresponding branch is not executed. The current test cases do include an addition operation, as well as the case with no items, but they are

failing to verify the conditions that arise when their list is empty. Thus, some branches will be left open and make it more difficult to achieve full branch coverage.

7. *continueFromTemp() in POSSystem*

In this case, statement coverage is impossible to reach at 100% since conditions for file handling cannot be tested and the test cases lack certain conditions. In addition, tests in place can test only against those cases that bring "Sale" or "Rental" as the output in the temporary log file- the "Return" case is not checked. Also, in tests given the line which deletes the file and logs the message is never executed if the file is empty. Any exception like FileNotFoundException or IOException thrown while reading the file will also cause statements following it to not be executed leaving code paths untested. All these reasons make the paths through the procedure untested in the current state, so some holes in statement coverage remain uncovered.

8. *updateTotal in PointOfSale*

The current test cases have holes; thus, it is impossible to achieve 100% statement coverage of the updateTotal function. The method calculates the value of the most recent item added to the transactionItem list to update the totalPrice. However, there is no output of item information and the running total from subsequent statements because the transactionItem list is empty; the line of code attempting to reference the last element throws an IndexOutOfBoundsException. Except for the alternative test for total with no items, the tests below do not include scenarios where the transactionItem list is empty. This would result in code paths not being tested and therefore those statements not getting executed.

9. *EndPOS() in POH class*

Because unexecuted paths can be followed through many events of the {endPOS} approach, a theoretically impossible 100% statement coverage is realized. In the second case, when variable {returnSale} is set to {false}, the block that writes to the return log file is fully skipped, and hence the statements are not executed. The third case is when statements that make the price calculations are not invoked when either {transactionItem} or {returnList} are empty. In this case, the total price update statements are also absent. Apart from these, if the input textFile is an empty string, then all the subsequent statements are skipped. Thus, there is no complete statement coverage in any of these cases.

Improvements: Suggest potential improvements or additional tests that could help achieve higher coverage.

1. *In POSSystem, there is login()*

Additional test cases will be developed to more clearly show that invalid username and invalid password are handled separately by defining the handling. Additionally, tests that test roles other than "Admin" and "Cashier" will ensure that all conditions are used so increasing coverage in the login method as a whole.

2. *retrieveTemp in the point of sale*

Mock file scenarios can be invented to exercise the behavior in the case where the tempFile doesn't exist or has content different from expected-for example, first line is

empty-to improve coverage of the retrieveTemp function. The overall coverage may be improved by creating tests simulating both FileNotFoundException and IOException besides others covering other branches of exception handling.

3. *Point of Sale updateTotal*

Test scenarios that specifically address an empty transactionItem list would add to the coverage of the updateTotal method in terms of ensuring that this method does not throw an exception if the situation occurs. It would also be good to write tests that check the output once items are added to the list to help ensure that all branches and conditions are validated.

4. *POSSystem's continueFromTemp()*

Testing for the continueFromTemp function should include cases that take on the "Return" scenario in the temporary log file, specifically. It would also be easier to ensure that all of the routes and exception handling scenarios are adequately tested if tests were written that dealt with both successful and failed file operations, such as deletions.

5. *Using the endPOS() of POH class.*

More test cases testing returnSale equals false should be added at the end to the POS method to explicitly test that the path writing to logs gets evaluated and coverage will be improved. Tests that would test the method's behavior in case that transactionItem, returnList or textFile is an empty collection would also help to cover the unexecuted pathways and improve statement coverage overall.

Element	Coverage	Covered Lines	Missed Lines	Total Lines
▼ CSE216	60.4 %	1,294	847	2,141
▼ src	44.8 %	645	794	1,439
▼ (default package)	44.8 %	645	794	1,439
> Management.java	83.2 %	149	30	179
> POSSystem.java	73.7 %	84	30	114
> EmployeeManagement.java	77.9 %	88	25	113
> POH.java	77.7 %	73	21	94
> POS.java	73.8 %	59	21	80
> POR.java	82.1 %	55	12	67
> Inventory.java	75.6 %	34	11	45
> PointOfSale.java	92.0 %	80	7	87
> Employee.java	100.0 %	10	0	10
> Item.java	100.0 %	8	0	8
> ReturnItem.java	100.0 %	5	0	5

2. Decision Coverage

Coverage Achieved: Explain where and why 100% coverage was possible.

Inventory class:

The linear control flow of the `Inventory` class, coupled with the apparent presence of branch points, make it readily obtain 100% branch coverage. Tests of both null and non-null

cases in the `getInstance()` method allow it to be easy to insure the `if` condition deciding whether the singleton instance is or isn't null. Simulating both successful and unsuccessful file reads (for example, file not found) test the `try-catch` block in the `accessInventory` method. Testing scenarios involving addition and subtraction of inventory items makes it easy enough to cover all the branches of the `updateInventory` function, including nested loops and `if` statements handling `takeFromInventory`. Such requirements allow covering all the branches with a big battery of tests.

Coverage Gaps: Describe any areas where 100% coverage was not achievable, including reasons for these gaps (e.g., limitations in the code, untestable scenarios).0

1. *continueFromTemp in POSSystem*

Because the `continueFromTemp` method has test cases of only very few small number that does not cover each scenario, it also fails to achieve 100% branch coverage. The tests as of now only check the branches for conditions where it returns "Sale" and "Rental" with the branch for "Return" not tested. Moreover, the tests coded so far do not exercise the following branches involving empty file scenarios or catch exception regarding file reading. Because the method returns an empty string if not all of the required conditions are satisfied and doesn't check the case in which the log file is invalid, many of the branches are not covered. As such, the approach can not have a full branch coverage because of these holes.

2. *Management() in Management class*

Since the checking for the environmental conditions in which the code is executed is not possible to a complete extent, it is also not possible to achieve 100% branch coverage in the following `{Management}` constructor. Because the conditional statement checks the name of the operating system with `System.getProperty("os.name")`, it produces two branches: the `if` branch for Windows-based systems and the `else` branch for non-Windows-based systems. Among these, one will not be executed except when performed as a test in both Windows and non-Windows environment. Moreover, full branch coverage is not achievable since it will be impossible to ensure that both the two branches will be executed during tests as code behavior will vary with setup.

Problems Javadoc Declaration Console Coverage × Call Hierarchy					
(default package) (Oct 7, 2024 8:44:01 PM)					
Element	Coverage	Covered Branches	Missed Branches	Total Branches	
▼ CSE216	48.8 %	204	214	418	
▼ src	46.1 %	167	195	362	
▼ (default package)	46.1 %	167	195	362	
> Management.java	79.6 %	43	11	54	
> POSSystem.java	73.3 %	22	8	30	
> POH.java	68.2 %	15	7	22	
> EmployeeManagement.java	82.4 %	28	6	34	
> PointOfSale.java	82.4 %	28	6	34	
> POS.java	62.5 %	10	6	16	
> POR.java	70.0 %	7	3	10	
> Employee.java		0	0	0	
> Inventory.java	100.0 %	14	0	14	
> Item.java		0	0	0	
> ReturnItem.java		0	0	0	

Improvements: Suggest potential improvements or additional tests that could help achieve higher coverage.

1. *continueFromTemp in POSSystem*

To cover the continueFromTemp method more thoroughly, even more tests checking each of the branches, in particular the "Return" case should be created as well. Because there are not any current ways to handle cases when the log file is either empty or invalid, tests should mimic those cases too. Moreover, so that all branches are covered by running tests, implement tests that call blocks that handle exceptions - namely FileNotFoundException and IOException. Mock file input may be much easier to test in various different scenarios compared with real files and could help ensure comprehensive coverage of all edge cases, thereby improving overall dependability.

2. *Management() in Management class*

Environmental checks' intrinsic limitations must thus be overcome to obtain 100% branch coverage for the Management constructor. Tests must be developed to check both branches of the conditional statement, due to the fact that the conditional statement in this case depends on the determination of whether a system is Windows or non-Windows. This may be accomplished either by continuous integration environments that simulate both conditions, or by a test framework that may play around with system attributes. Add a config option that may then identify OS type at test time, thereby ensuring that both are exercised - a configuration option that may specifically identify the OS type

at test time, thereby enabling further testing of environmental dependencies.

3. Condition Coverage

Coverage Achieved: Explain where and why 100% coverage was possible.

addRental in Management class

Because the conditionals in the addRental method involve such typical conditional logic, all pathways are reachable within this method and 100% condition coverage is possible. The main conditional to test for the procedure is that the value passed in the phone argument equals the nextPhone variable. Test the branch for this method by passing different values of phone and both match and non-match values for complete coverage of true and false. Application of exception handling surrounding the operations of files is also very basic, and an approach would be about handling both FileNotFoundException and IOException to ensure proper analysis of all possible cases as it accesses and modifies the file of relevance to the user database.

Coverage Gaps: Describe any areas where 100% coverage was not achievable, including reasons for these gaps (e.g., limitations in the code, untestable scenarios).

addRental in Management class

Even under 100% condition coverage, it can be challenging to obtain complete statement coverage at all places. For instance, if the userDatabase file is empty or formatted incorrectly, the file line processing algorithm will not work as anticipated. It does not also stress test its catch block when it throws a NumberFormatException due to bad data; instead, it only skips to the next iteration with no hint on handling or logging. These instances are untested branches or

unexecuted statements-mostly the file reading and writing logic- and thus leave holes in coverage.

Improvements: Suggest potential improvements or additional tests that could help achieve higher coverage.

addRental in Management class

For more coverage, several modifications might be done. For instance, to test routes that have not been covered currently, it would be helpful to construct unit tests that particularly handle edge cases such as when the file `userDatabase` might be empty or incorrectly formatted. Mock objects also can be used to simulate file I/O operations so that various scenarios could be tested while not depending on real files. Along with this aspect, improved error reporting in the catch blocks would also ensure that certain code paths are run during testing. If the file handling logic is broken down into more fragmented, tested methods, testing may be simplified and coverage increased.

a. Generate and Submit Reports:

- i. **JUnit Test Report:** Capture and include screenshots of the JUnit test results showing which tests were executed and their outcomes.

```

  ManagementTest [Runner: JUnit 4] (0.130 s)
    ✓ testCheckUser_UserExists (0.000 s)
    ✓ testCheckUser_UserDoesNotExist (0.000 s)
    ✓ testCreateUser_ValidPhone (0.001 s)
    ✓ testAddRental_SingleItem (0.076 s)
    ✓ testUpdateRentalStatus_SingleItem (0.027 s)
    ✓ testAddRental_MultipleItems (0.010 s)
    ✓ testUpdateRentalStatus_MultipleItems (0.011 s)
    ✓ testGetLatestReturnDate_ValidPhone (0.005 s)
  PORTest [Runner: JUnit 4] (0.033 s)
    ✓ testDeleteTempItem_1005 (0.006 s)
    ✓ testEndPOS_ItemDatabase (0.007 s)
    ✗ testRetrieveTemp_1002 (0.012 s)
    ✗ testRetrieveTemp_1005 (0.005 s)
    ✓ testDefaultConstructor (0.003 s)
  POSSystemTest [Runner: JUnit 4] (0.071 s)
    ✓ testCheckTemp_True (0.011 s)
    ✓ testLogin_InvalidPassword (0.006 s)
    ✗ testLogin_ValidAdmin (0.006 s)
    ✗ testContinueFromTemp_Rental (0.012 s)
    ✓ testContinueFromTemp_Return (0.007 s)
    ✗ testContinueFromTemp_Sale (0.006 s)
    ✓ testLogout (0.006 s)
    ✓ testLogin_InvalidUsername (0.006 s)
    ✓ testLogin_ValidCashier (0.006 s)
    ✓ testCheckTemp_False (0.005 s)

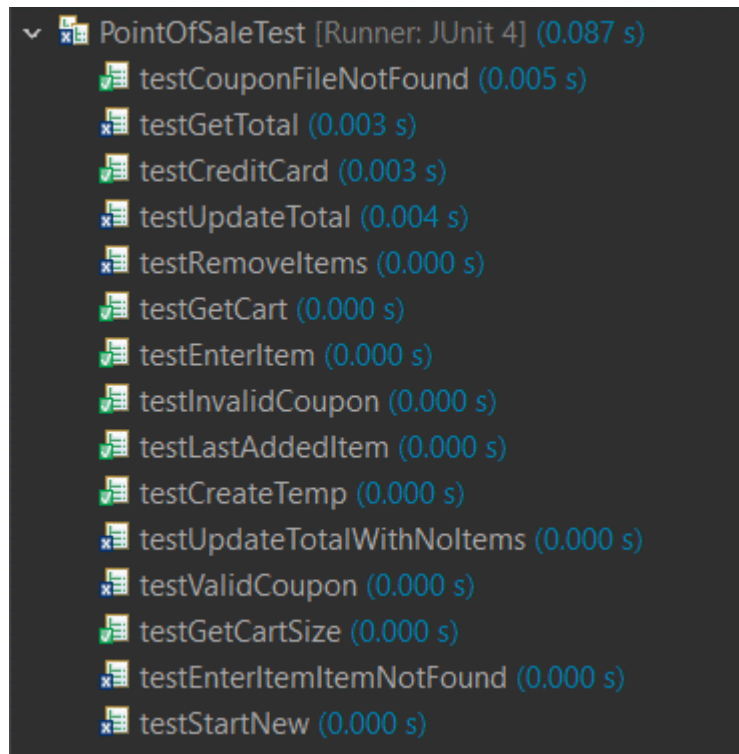
```

```

  ✓ POSTest [Runner: JUnit 4] (0.040 s)
    ✓ testEndPOS_ItemDatabaseUpdate (0.006 s)
    ✓ testEndPOS_ReturnSaleUpdate (0.005 s)
    ✓ testEndPOS_SumCheck (0.004 s)
    ✓ testDeleteTemplItem (0.007 s)
    ✗ testRetrieveTemp_AmountCheck1002 (0.005 s)
    ✗ testRetrieveTemp_AmountCheck1005 (0.004 s)
    ✗ testRetrieveTemp_TotalPriceUpdate (0.006 s)
    ✓ testDefaultConstructor (0.003 s)
  ✓ POHTest [Runner: JUnit 4] (0.049 s)
    ✗ testDeleteTemplItem_1002 (0.007 s)
    ✗ testDeleteTemplItem_1005 (0.005 s)
    ✓ testEndPOS_UpdatedReturnSale (0.006 s)
    ✓ testEndPOS_UpdatedItemDatabase (0.005 s)
    ✗ testRetrieveTemp_Item1002 (0.004 s)
    ✗ testRetrieveTemp_Item1005 (0.008 s)
    ✓ testEndPOS_CorrectTotal (0.009 s)
    ✓ testDefaultConstructor (0.003 s)
  ✓ ItemTest [Runner: JUnit 4] (0.002 s)
    ✓ testItemPrice (0.001 s)
    ✓ testupdateAmount (0.000 s)
    ✓ testAmount (0.000 s)
    ✓ testItemID (0.000 s)
    ✓ testItemName (0.000 s)
  ✓ ReturnItemTest [Runner: JUnit 4] (0.002 s)
    ✓ testGetItemID_10001 (0.001 s)
    ✓ testGetItemID_10002 (0.000 s)
    ✓ testGetDays_5 (0.001 s)
    ✓ testGetDays_12 (0.000 s)

```

- ✓ EmployeeManagementTest [Runner: JUnit 4] (0.020 s)
 - ✓ testDelete_NonExistingEmployee (0.003 s)
 - ✓ testDelete_ExistingEmployee (0.006 s)
 - ✓ testUpdate_InvalidPosition (0.003 s)
 - ✓ testAdd_Cashier (0.002 s)
 - ✓ testUpdate_ValidUpdate (0.003 s)
 - ✓ testAdd_Admin (0.002 s)
 - ✓ testUpdate_InvalidUsername (0.001 s)
- ✓ InventoryTest [Runner: JUnit 4] (0.014 s)
 - ✓ testUpdateInventorySale_Potato (0.003 s)
 - ✓ testUpdateInventoryReturn_SkirtSteak (0.002 s)
 - ✓ testUpdateInventorySale_PlasticCup (0.002 s)
 - ✓ testUpdateInventoryReturn_PotatoChips (0.003 s)
 - ✓ testGetInstancelNotNull (0.001 s)
 - ✓ testAccessInventory (0.002 s)
 - ✓ testGetInstance (0.001 s)
- ✓ EmployeeTest [Runner: JUnit 4] (0.001 s)
 - ✓ testSetPosition_10006 (0.001 s)
 - ✓ testGetUsername_10001 (0.000 s)
 - ✓ testGetPosition_10003 (0.000 s)
 - ✓ testSetPassword_10007 (0.000 s)
 - ✓ testGetName_10002 (0.000 s)
 - ✓ testGetPassword_10004 (0.000 s)
 - ✓ testSetName_10005 (0.000 s)
- ✗ PointOfSaleTest [Runner: JUnit 4] (0.087 s)
 - ✓ testCouponFileNotFound (0.005 s)
 - ✗ testGetTotal (0.003 s)
 - ✓ testCreditCard (0.003 s)
 - ✗ testUpdateTotal (0.004 s)
 - ✗ testRemoveItems (0.000 s)
 - ✓ testGetCart (0.000 s)
 - ✓ testEnterItem (0.000 s)
 - ✓ testInvalidCoupon (0.000 s)
 - ✓ testLastAddedItem (0.000 s)
 - ✓ testCreateTemp (0.000 s)



- ii. **Coverage Report:** Capture and include screenshots of the coverage report generated by Eclemma.

1. *Statement Coverage Report*

Element	Coverage	Covered Lines	Missed Lines	Total Lines
▼ CSE216	60.4 %	1,294	847	2,141
▼ src	44.8 %	645	794	1,439
▼ (default package)	44.8 %	645	794	1,439
> Management.java	83.2 %	149	30	179
> POSSystem.java	73.7 %	84	30	114
> EmployeeManagement.java	77.9 %	88	25	113
> POH.java	77.7 %	73	21	94
> POS.java	73.8 %	59	21	80
> POR.java	82.1 %	55	12	67
> Inventory.java	75.6 %	34	11	45
> PointOfSale.java	92.0 %	80	7	87
> Employee.java	100.0 %	10	0	10
> Item.java	100.0 %	8	0	8
> ReturnItem.java	100.0 %	5	0	5

2. Decision Coverage Report

Problems Javadoc Declaration Console Coverage × Call Hierarchy					
(default package) (Oct 7, 2024 8:44:01 PM)					
Element		Coverage	Covered Branches	Missed Branches	Total Branches
▼ CSE216		 48.8 %	204	214	418
▼ src		 46.1 %	167	195	362
▼ (default package)		 46.1 %	167	195	362
> Management.java		 79.6 %	43	11	54
> POSSystem.java		 73.3 %	22	8	30
> POH.java		 68.2 %	15	7	22
> EmployeeManagement.java		 82.4 %	28	6	34
> PointOfSale.java		 82.4 %	28	6	34
> POS.java		 62.5 %	10	6	16
> POR.java		 70.0 %	7	3	10
> Employee.java			0	0	0
> Inventory.java		 100.0 %	14	0	14
> Item.java			0	0	0
> ReturnItem.java			0	0	0